

验证报告 / 2026-09-05

Mini-Drop AI 诊断 与 Skill 复用测试报告

面向面试演示的功能闭环、数据集、质量门禁与证据边界

540	93.70%	307	68
新设计盲测 Case	Skill 路由通过率	Python 回归通过	OpenAPI 路由对齐
公开 query / 私有 oracle	506/540	另有 3 项跳过	method/path pairs

报告结论 页面演示链已经从“单句触发 + 静态回放”升级为可继续对话、可人工改变调查方向、可实时观察探索树、可启动白名单故障并发起真实 Skill A/B 的诊断工作台。

重要边界 93.70% 是 Skill 路线选择与拒绝的静态盲测结果，不是根因准确率；本机 Docker Linux daemon 未运行，因此本报告不声称完成 Linux live E2E。

版本范围 D:\tx\mini-drop 当前未提交工作树；指标由 benchmark-report.json 读取。

Evidence before conclusion. Route memory before repeated exploration.

1. 执行摘要

这轮改造解决的不是三个孤立的页面 Bug，而是三条演示链断点：范围确认缺少幂等语义、采集成功与样本有效性混在一起、以及诊断会话缺少可见的多轮交互。修复后的系统把用户输入视为调查上下文，把 Artifact 经过质量门禁后形成 Evidence，再由报告引用 Evidence 给出可验证结论。

用户看到的问题	根因	现在的行为
重复确认范围时报 immutable	浏览器分钟精度与服务端秒/时区表达直接比较	同一 UTC 语义与页面分钟重试幂等；真实移动时间窗仍拒绝
Task DONE 却没有火焰图	perf header-only 也可能以退出码 0 结束	零样本、空折叠栈、空可视化 fail-closed，并返回可行动重采提示
总是证据不足或一直准备	无受控故障、无后续回合、动态树藏在回放弹窗	故障广场 + 多轮干预 + 主页面动态树 + 可继续调查
Skill 复用无法现场对比	旧页面只有离线或硬编码指标	AUTO/DISABLED 创建两条真实 Diagnosis，比较真实 ID、Tool、Evidence、Report

验收判断

- 功能合同：后端、OpenAPI、故障白名单、时间窗幂等和干预事件已有自动化覆盖。
- 页面闭环：用户可以从故障广场进入诊断、在诊断中继续追问/质疑/换方向，并在同页观察探索树 revision。
- 采集可信度：以后不会再把 0 KB JSON 和空 SVG 当成成功火焰图；历史空产物需重新采集。
- 实验可信度：540-case 可以证明路由选择器的行为，不能替代真实 Linux 故障的根因评测。

2. 页面演示闭环

面试演示时，推荐按下面的因果顺序讲，而不是按菜单逐页点击：

故障广场 → Diagnosis → Hypothesis → Tool Call → Task / Artifact → Evidence → Report → 恢复验证

页面能力	现场动作	面试官能看到的真实性信号
故障广场	选择 CPU、源码热点、内存、I/O、噪声邻居、负载或队列场景，限定时长并启动	浏览器只提交 scenario_id 与 duration；服务端固定白名单，不接收任意命令/PID/URL
AI 诊断	描述症状并让 AI 自动发现安全目标，或人工确认目标	Diagnosis ID、绑定 ID、时间窗、Skill policy 都由服务端持久化
多轮对话	补充上下文、质疑假设、换方向、继续调查	每次提交产生 intervention 事件；用户话语不会直接成为 Evidence

页面能力	现场动作	面试官能看到的真实性信号
动态探索树	观察假设、探针、任务、证据和报告节点实时增加	revision/version 增长，方向切换与人工节点可追溯
Skill A/B	同问题分别创建 DISABLED 与 AUTO 会话	对比两条真实会话的 Skill activation、Tool、Evidence、Report 和耗时
任务结果	进入采集 Task 查看 Artifact 和可视化	空样本明确失败；有样本才渲染火焰图和 TopN

人工干预的四种语义

动作	用途	安全语义
ADD_CONTEXT	补充部署变更、症状、业务背景	只作为 Context，不升格为事实证据
CHALLENGE_HYPOTHESIS	要求 AI 反驳当前主假设	保留旧假设历史，创建新的可证伪方向
CHANGE_DIRECTION	从 CPU 转向 I/O、内存、网络或运行时	新探针仍经过目标绑定、能力、风险和预算门禁
CONTINUE_INVESTIGATION	证据不足或已有结论后继续一轮	旧报告不可变，新事件进入下一轮调查

3. 540-case Skill 路由盲测

数据集使用固定种子 20260905 生成 540 条新 query。公开输入与私有 oracle 分开保存；生成器不读取已退役的旧 Case，但故障分类和正确 Skill 来自当前 catalog。因此它是可复现的项目内盲测，不是外部独立数据集。

92.78% 正例复用 334/360	95.56% 负例拒绝 172/180	4.44% 负例误激活 8/180
--	--	--

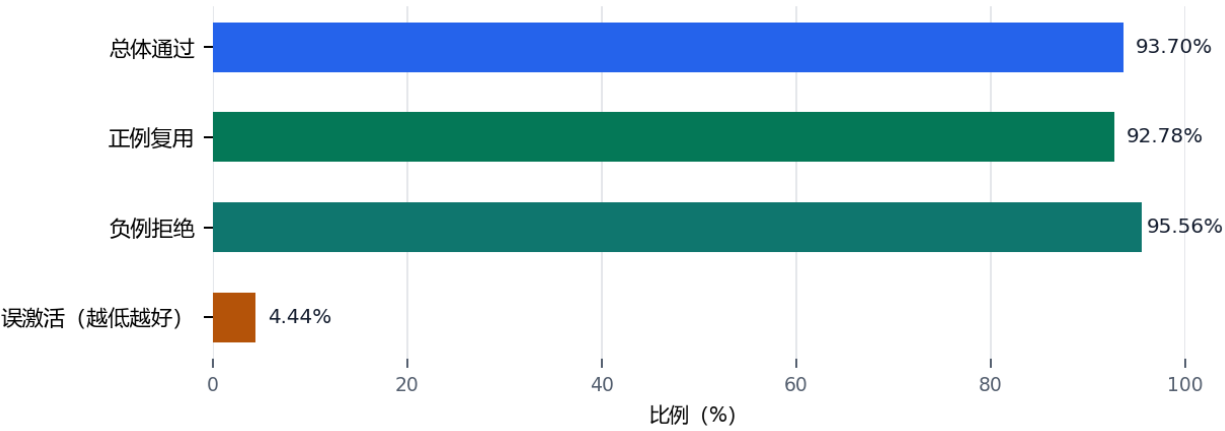


图 1 Skill 路由选择与安全拒绝指标 (确定性重跑)

Case 家族	数量	Skill 通过	No-Skill 基线	测试意图
POSITIVE_REUSE	360	334/360 (92.78%)	0/360	描述足够具体时命中正确 Skill
MISLEADING_OR_UNDESPECIFIED	90	82/90 (91.11%)	90/90	歧义或信息不足时应弃权
CAPABILITY_OR_ENVIRONMENT_DRIFT	90	90/90 (100%)	90/90	环境/能力不匹配时拒绝复用

指标解释

- 总体 506/540 (93.70%)：正例选择指定 Skill，负例正确弃权。
- No-Skill 基线 180/540 (33.33%)：它不是另一套 RCA Agent，只是永远不复用路线的保守下界。
- 8 个误激活全部来自误导/信息不足负例；它们是下一轮阈值和歧义门禁优化的直接样本。
- Skill 命中代表可复用的调查先验，不代表根因已被证明。

4. 火焰图为空：修复与质量门禁

原问题的关键不在前端渲染。perf script 和折叠脚本可能对 header-only 或 0 样本输入返回退出码 0；旧 Analyzer 随后生成空 root、空 TopN 和极小 SVG，任务仍被标为 DONE。新链路同时检查进程退出状态和内容质量。

检查点	拒绝条件	页面结果
perf script	没有任何可解析 sample	NO_PERF_SAMPLES；提示确认 PID、权限、采样时长和目标是否在运行
stack collapse	没有正权重 folded stack	NO_FOLDED_STACKS；不生成伪火焰图
Analyzer 输出	sample_count=0、profile_quality=UNUSABLE、可视化 0 bytes	分析失败并保留结构化原因

检查点	拒绝条件	页面结果
Artifact 合同	新产物明确声明 0 样本或不可用	阻止进入 Evidence；旧产物缺字段仅标记质量未知

必须重采 截图中的历史空 flamegraph.json / flamegraph.svg 不会被“修复成有内容”。修复保证新任务诚实失败；要看到火焰图，必须对一个确实消耗 CPU 的目标在当前 Analyzer 下重新采集。

5. 自动化验证结果

验证层	结果	覆盖内容	结论
Python 全量回归	307 passed / 3 skipped	诊断、策略、分析器、合同、故障广场、时间窗、干预树	通过
OpenAPI 对齐	68 method/path pairs	Python RPC 实现与公开契约	通过
静态 Skill 盲测	506/540	生产混合检索与拒绝门禁	93.70%
受控故障本地烟测	list/start/stop	白名单 demo 状态、启动、停止与诊断请求	通过
Linux live E2E	本机未执行	真实 Agent、perf 权限、Artifact、Evidence、Report	待云端 Campaign

三层测试模型

- 静态盲测：快速验证“该不该复用哪条路线”，适合放入 CI。
- 受控故障合同/集成：验证白名单、状态机、幂等、人工干预、质量错误和事件投影。
- Linux live E2E：在真实 Worker 和标注故障下，对比根因服务/指标、首条有效 Evidence 耗时、Tool Call 数、空 Profile 率和恢复验证。

本报告完成前两层。第三层需要 Linux Worker、稳定负载、故障重置和根因真值；Windows 上 Docker Desktop Linux Engine 未运行，不能用静态截图冒充当次 live 证据。

6. 面试演示脚本（8~10 分钟）

- 打开 AI 诊断中的故障广场，选择“CPU 计算热点”，设置 120 秒并启动。说明浏览器不能下发任意命令。
- 点击“启动并诊断”，让 AI 自主选择服务端签发的安全目标和时间窗；展示 Diagnosis ID 与绑定 ID。
- 用 DISABLED 创建对照臂，记录第一个 Tool Call。恢复并重放同一故障，再用 AUTO 创建 Skill 臂。
- 在运行中的会话输入“CPU 样本没有主导热点，请质疑当前假设并转查 I/O 等待”。展示 intervention 节点和树 revision 增长。

- 5. 沿 Tool Call 打开 Task，查看 Attempt、Artifact、Analyzer 和 Evidence。如果样本为空，现场展示结构化质量错误与重采路径。
- 6. 打开 Skill A/B 面板，对比两条真实会话的 Skill activation、工具数、首条 Evidence、状态、报告版本和耗时。
- 7. 停止故障并验证服务恢复。最后打开 540-case 报告，明确它只测路线复用，不等于刚才的 live 根因链。

演示时建议主动说出的边界

- 人工输入是 Context，只有 Collector/Analyzer 产物通过质量门禁后才能成为 Evidence。
- Skill 是经过验证的调查路线记忆，不是把历史结论直接复制到新事故。
- A/B 的两个臂必须重置同一故障和负载；否则顺序运行会被时间漂移污染。
- INSUFFICIENT_EVIDENCE 是安全结论，不是系统卡死；用户可以继续调查，但模型不能用猜测补齐证据。

7. 已知限制与下一步验收

限制	影响	下一步
本机没有 Linux live E2E	无法声明根因服务/指标准确率	在云端 Worker 运行固定种子的受控故障 Campaign
故障广场主要是单机/单进程	对跨服务因果链覆盖有限	接入 OpenTelemetry Demo 级微服务场景
live A/B 脚本不负责重置故障	顺序臂可能受状态漂移影响	由 Campaign orchestrator 在每臂前重置并校验负载
旧 Profile 缺少质量元数据	兼容读取时只能标记质量未知	用新 Analyzer 重算或重新采集
误导负例仍有 8 次误激活	Skill 可能在信息不足时过早介入	基于失败 Case 调整阈值并保留独立回归集

8. 复现命令与证据文件

```
python -m pytest -q
python scripts/check_openapi_routes.py
python scripts/generate_diagnosis_benchmark_v2.py
python scripts/run_diagnosis_benchmark_v2.py
docker compose --profile demo-target up -d --build
```

证据	位置
公开盲测输入	benchmarks/diagnosis-v2/public/cases.json
私有 oracle	benchmarks/diagnosis-v2/private/oracles.json
机器可读结果	web/public/report-assets/skill-evolution/benchmark-report.json
人类可读报告	reports/ai-diagnosis/AI 诊断与 Skill 复用测试报告-20260905.md

证据	位置
核心设计文档	docs/AI_DIAGNOSIS.md、docs/SKILLS.md、docs/PROJECT_LEARNING_GUIDE.md

参考依据

- Microsoft AIOpsLab — application/task/fault/workload/evaluator 分层与 action trace 评估：
<https://github.com/microsoft/AIOpsLab>
- AIOpsLab paper: https://www.microsoft.com/en-us/research/wp-content/uploads/2024/10/arxiv_AIOpsLab.pdf
- RCAEval — 以标注故障分别评估 root-cause service 与 indicator: <https://github.com/phamquillan/RCAEval>
- RCAEval paper: <https://arxiv.org/abs/2412.17015>
- OpenTelemetry Demo feature flags — 可重放问题场景: <https://opentelemetry.io/docs/demo/feature-flags/>
- Chaos Mesh — 有 selector、duration 和恢复语义的受控故障: <https://chaos-mesh.org/docs/run-a-chaos-experiment/>

结论必须能沿 Report → Evidence → Artifact → Attempt → Task 反向追溯。
